

# 情報リテラシー 第13回「デジタル表現1：2進数 / 文字コード / 音」

2026 年度 稲積 泰宏 / 教科書 15-17

## 今日の流れ

- ① 2進法の加算・補数・減算 (Theory 15)
- ② 文字コードとフォント (Theory 16)
- ③ 音のデジタル化 (Theory 17)

## 今日の目標

- 2進法の加算と補数の仕組み, 文字コード, 音声のデジタル化について説明できる。

## キーワード

補数 / 文字コード / **ASCII** / ユニコード (**Unicode**) / 文字化け / フォント / 標本化 (サンプリング) / 量子化 / 符号化 / 標本化定理

# 2進法の加算

## ポイント

- 2進法の加算は 10進法と同様に桁ごとに計算する
- 違いは  $1 + 1 = 10_{(2)}$  (繰り上がり) となること

## 例題

- ①  $0100 + 0001 = 0101$
- ②  $0101 + 1001 = 1110$

## 解説

2進法の加算は 10進法と同じ手順だが、「2 になったら繰り上がる」のがポイント。

① :  $0 + 1 = 1$ ,  $0 + 0 = 0$ ,  $1 + 0 = 1$ ,  $0 + 0 = 0 \rightarrow 0101$ 。  
② :  $1 + 1 = 10_{(2)}$  (繰り上がり),  $0 + 0 + 1 = 1$ ,  $1 + 0 = 1$ ,  $0 + 1 = 1 \rightarrow 1110$ 。繰り上がりに注意しながら桁ごとに計算しよう。

# 補数と減算

## 補数 (2 の補数)

- 補数：ある自然数に足して「切り上がる最も小さな数」
- 定義： $n$  ビットの 2 の補数 =  $2^n$  から引いた数
- 計算法：全ビットを反転して 1 を加える（結果は同じ）
- 例：4 ビットの 1011 の補数  $10000 - 1011 = 0101$  / 反転  $\rightarrow 0100$ ,  $+1 \rightarrow 0101$

## 補数を使った減算

- 2 進法の減算は補数を使って加算に置き換えられる
- 例： $0101 - 0010$  を計算する
- 0010 の補数 ( $10000 - 0010$ ) = 1110
- $0101 + 1110 = 10011 \rightarrow$  最上位ビットを捨てて **0011**

## 解説

補数の定義：「足して切り上がる最も小さな数」。4 ビットなら  $2^4 = 10000$  から引く。  
計算ショートカット：全ビット反転して+1（結果は同じ）。補数を使うと引き算が足し

# コンピュータでの整数の表現

## ポイント

- 2 の補数を使うと、1 種類のビット列で正・負の整数を表現できる
- 4 ビットでは  $-8 \sim +7$  の 16 種類の整数を表現できる
- 最上位ビット (MSB) が **0**  $\rightarrow$  正の数, **1**  $\rightarrow$  負の数

| 2 進数 | 10 進数 |
|------|-------|
| 0000 | 0     |
| 0001 | 1     |
| 0010 | 2     |
| 0011 | 3     |
| 0100 | 4     |
| 0101 | 5     |
| 0110 | 6     |
| 0111 | 7     |

| 2 進数 | 10 進数 |
|------|-------|
| 1000 | -8    |
| 1001 | -7    |
| 1010 | -6    |
| 1011 | -5    |
| 1100 | -4    |
| 1101 | -3    |
| 1110 | -2    |
| 1111 | -1    |

## 解説

MSB が 0 なら正, 1 なら負。4 ビットの場合, 正側は 0000~0111 (0~7), 負側は 1000~1111

# 文字コードとフォント

## 文字コード

- 文字コード：文字と数値（ビット列）の対応規則
- **ASCII**：英数字・記号を 7 ビット（128 文字）で表す。ほぼすべての文字コードの基礎
- **JIS8 ビットコード**：**ASCII** を 8 ビットに拡張し、カタカナを追加（256 種）
- **シフト JIS コード**：JIS コードを改良し、漢字・ひらがなを 2 バイトで表現
- **ユニコード（Unicode）**：世界中の文字を統一的に扱える国際標準（UTF-8 等）
- 文字化け：異なる文字コードで読み込むと文字が正しく表示されない現象

## フォントの種類

- **ビットマップフォント**：文字をドット（画素）の集合で表現。拡大すると粗くなる
- **アウトラインフォント**：文字の輪郭を数式で表現。拡大しても滑らかに表示

# 音のデジタル化

## ポイント

- 音はアナログ（空気の連続的な振動）→ A/D 変換でデジタル化
- 音を特徴づける要素：振幅（音の大きさ）・周波数（音の高低）・波形（音色）
- ①標本化（サンプリング）：一定時間間隔で波の高さを取り出す
- ②量子化：取り出した値を一定の段階値に丸める
- ③符号化：量子化した数値を 2 進法で表す

## 解説

音のデジタル化の 3 ステップ：①標本化→②量子化→③符号化。CD の規格：標本化 44,100Hz（1 秒間に 44,100 回）、量子化 16 ビット（65,536 段階）。人間の可聴域は 20～20,000Hz で、ナイキスト定理よりその 2 倍以上（40,000Hz 以上）でサンプリングする必要がある→ 44,100Hz。

# 音のデータ量計算

## ポイント

- 1 秒間のデータ量 (bit) = 標本化周波数 (Hz) × 量子化ビット数 (bit) × チャンネル数
- 例：標本化周波数 24,000Hz, 量子化 16bit, ステレオ (2ch) の場合
- $24,000 \times 16 \times 2 = 768,000$  bit/秒 (= 768 kbit/秒)
- 品質↑ (標本化周波数・量子化ビット数を増やす) → データ量↑ (トレードオフ)

## 標本化定理

- 元の信号の最大周波数の **2 倍以上** の周波数でサンプリングすれば、元の信号を正確に復元できる

## 解説

計算手順：  $24,000 \times 16 \times 2 = 768,000$  bit/秒。1 分間：  $768,000 \times 60 = 46,080,000$  bit = 5,760,000 B = 5,760 kB。標本化定理：人間の可聴域 20,000Hz の 2 倍=40,000Hz 以上→ CD は 44,100Hz でサンプリング。品質を高める (周波数・ビット数を上げる) とデータ量が増えるトレードオフに注意。

# 演習

各自で取り組んでください。

## 演習 1

4 ビットの 2 進数 0110 の補数を求めよ

## 演習 2

ASCII とユニコードの違いを説明せよ

## 演習 3

「標本化」「量子化」「符号化」を、音のデジタル化を例にして説明せよ

## 解説

【解答例】演習 1 :  $10000 - 0110 = 1010$  (または反転  $\rightarrow 1001$ ,  $+1 \rightarrow 1010$ )。演習 2 : ASCII は英数字 128 文字のみ (7 ビット), Unicode は世界中の文字を統一的に表現 (UTF-8 等で可変長エンコード, 14 万字以上)。演習 3 : 音の波形を一定間隔で取り出す (標本化)  $\rightarrow$  段階値に丸める (量子化)  $\rightarrow$  2 進数に変換 (符号化)。



# まとめ・チェックリスト

## 自己チェック（できたら ✓ をつけよう）

- ☐ 2進法の加算と補数（負の数・減算への応用）を習得した
- ☐ 文字コード（ASCII・Unicode）と文字化けの原因，フォントの種類を理解した
- ☐ 音のデジタル化（標本化・量子化・符号化）とデータ量計算を学んだ

質問があれば授業後または Teams で受け付けます。